

Morse Code Transmitter Project Report

Jokobus

Github

Technical University of Github

Somewhere, Around

Abstract—This report describes an interrupt-driven Morse code transmitter built with an Arduino Nano (ATmega328P) and a WS2812B LED strip. The system converts text messages into Morse code and displays them using a lighthouse-style wave effect across the LEDs. The implementation uses Timer1 hardware interrupts for precise timing, EEPROM for state persistence of button presses, and demonstrates key embedded systems concepts like interrupt service routines and timer configuration. A major achievement is that the main loop is kept free of any code.

Index Terms—Embedded Systems, Arduino, Morse Code, WS2812B, Timer Interrupts, ATmega328P, EEPROM

I. INTRODUCTION

A. Background

Morse code was developed by Samuel Morse in the 1830s as one of the first digital communication methods. It encodes text as sequences of dots (short signals) and dashes (long signals), which was perfect for telegraph systems. While we have modern digital protocols now, Morse code is still used in amateur radio, aviation, and maritime communications. It's also a great way to learn about digital signal processing and timing in embedded systems.

B. Project Motivation

The goal was to build a visual Morse code transmitter that goes beyond simple LED blinking. This project features:

- Using interrupts for accurate timing to keep the main program loop free
- Converting string messages to Morse code dynamically
- Remembering the on/off state across power cycles
- Controlling addressable RGB LEDs with precise timing

C. System Overview

The system reads a predefined message string, converts it to Morse code, and displays it on an 8-LED WS2812B strip with an optional sweeping lighthouse effect. The message loops continuously until manually stopped by pressing the reset button, which toggles between playing and stopped states. The implementation prioritizes timing accuracy, code modularity, and visual appeal.

II. HARDWARE ARCHITECTURE

A. Microcontroller Selection

I chose the Arduino Nano with its ATmega328P microcontroller because it has:

- A 16 MHz clock that gives enough timing resolution

- Three hardware timers (Timer0, Timer1, Timer2) for generating precise interrupts
- 1 KB EEPROM to store data when powered off
- A compact size that's good for embedded projects
- 32 KB flash memory, which is plenty for the program and Morse lookup tables
- And most importantly: It was already in my drawer

The ATmega328P runs at 5V, which works well with most LED strips and can drive the data pin directly.

B. LED Strip Technology

The WS2812B LEDs are addressable RGB LEDs with a built-in driver chip in each LED. This means you can control each LED's color individually. Important specs:

- **Protocol:** Single-wire serial communication
- **Color depth:** 24-bit RGB (8 bits per channel)
- **Timing requirements:** Very precise pulse widths ($0.4\mu s$ and $0.8\mu s$)
- **Data rate:** 800 kHz
- **Cascading:** Data passes through from one LED to the next

The strip connects to digital pin 6. Each LED draws about 60 mA at full white brightness, but I'm running them at just 1% brightness (2.55/255) because the LEDs intensity is quite high – especially if one looks directly into them by accident.

C. Hardware Connections

The Arduino is mounted on a prototype shield (Nano expansion board) that provides convenient access to all pins which makes it easier to connect the LED strip without soldering directly to the Arduino pins. It also provides a more robust mounting solution and better access to power rails for future expansions.

- **LED Data Pin:** Digital Pin 6 (PD6)
- **Power Supply:** 5V from USB or external source
- **Ground:** Common ground between Arduino and LED strip
- **LED Count:** 8 individually addressable LEDs

The reset button on the Arduino Nano serves as the user interface for toggling the transmission state, demonstrating a minimalist approach to user interaction. Using an external button would have been a cleaner approach, but I don't have one. However, the EEPROM solution works quite well.

III. SOFTWARE ARCHITECTURE

A. System Design Philosophy

The software follows an interrupt-driven architecture where the main timing loop executes independently of the `loop()` function. This design ensures:

- Timing accuracy unaffected by main loop execution time
- Non-blocking operation allowing future feature additions
- Deterministic behavior for real-time signal generation
- Efficient CPU utilization

B. Design Decisions

Here are some key decisions I made:

Why Timer1: I chose Timer1 over the other timers because:

- Timer0 is already used by Arduino's `delay()` and `millis()` functions
- Timer1 is 16-bit, so it can handle longer time intervals easily
- Timer2 is only 8-bit

Using Interrupts: Interrupts keep the timing accurate no matter what the main loop is doing. This means I could add features like serial communication later without messing up the Morse timing.

Lighthouse Effect: The wave effect makes it look cooler than just blinking all LEDs on and off. It also shows more fine-grained control over the individual LEDs.

C. Morse Code Timing Standards

The timing follows standard Morse code ratios but with longer durations for better visual clarity:

- **Dot:** 500 ms
- **Dash:** 1500 ms (3× dot duration)
- **Intra-character gap:** 500 ms (between dots/dashes)
- **Inter-character gap:** 1500 ms (between letters)
- **Word gap:** 3500 ms (between words)

I used 500 ms for dots instead of the typical 200 ms because faster flashing would be hard to see clearly. The slower timing makes it easier to follow the message visually.

D. Message Encoding

The `morseCode()` function is basically a lookup table that converts characters to Morse code:

- All 26 letters (A-Z)
- All 10 digits (0-9)
- Works with both uppercase and lowercase (converts everything to uppercase)
- Handles spaces between words

The function returns a pointer to a string of dots and dashes. Since these strings are constant, they're stored in flash memory instead of RAM, which saves memory.

E. Sequence Builder

The `buildSequence()` function creates the complete animation sequence:

- 1) Goes through each character in the message
- 2) Converts to uppercase (so it's case-insensitive)
- 3) Looks up the Morse pattern (dots and dashes)
- 4) Breaks each dot/dash into individual LED steps for the wave effect
- 5) Adds the right timing gaps between symbols, letters, and words
- 6) Adds a reverse wave at the end for visual effect

The sequence is stored in three arrays:

- `durations[]`: How long each step lasts (in milliseconds)
- `colors[]`: The RGB color for each step
- `ledIndex[]`: Which LED to light (255 means all LEDs)

With `MAX_STEPS` set to 100, this handles messages of about 10-15 characters. If you need longer messages, just increase `MAX_STEPS`.

F. Memory Management

Efficient memory usage is critical on microcontrollers with limited RAM. The ATmega328P has only 2 KB of SRAM, so careful planning was needed:

Flash vs. RAM Storage:

- The Morse lookup table uses string literals stored in flash memory (32 KB available) rather than RAM
- The message string is also stored in flash using the `const char[]` declaration
- Only the sequence arrays (`durations[]`, `colors[]`, `ledIndex[]`) occupy RAM during runtime

RAM Usage Breakdown:

- `durations[100]`: 200 bytes (uint16_t)
- `colors[100]`: 400 bytes (uint32_t)
- `ledIndex[100]`: 100 bytes (uint8_t)
- LED strip object: 150 bytes
- Stack and other variables: 200 bytes
- **Total:** 1050 bytes (52% of available RAM)

This leaves enough headroom for future features while keeping the current implementation stable.

G. Lighthouse Wave Effect

Instead of just turning all LEDs on/off together, I implemented a lighthouse wave effect where the light sweeps across the strip:

- 1) The dot or dash duration is split equally among the 8 LEDs
- 2) LEDs light up one after another from position 0 to 7
- 3) `WAVE_WIDTH` controls how many neighboring LEDs light up at once
- 4) Boundary checking keeps the indices from wrapping around the strip

For example, a 500 ms dot divided by 8 LEDs means each LED stays on for 62.5 ms. This creates a smooth sweeping effect that looks much better than simple blinking. The standard

blinking without the lighthouse effect is supported as well by setting `WAVE_WIDTH = 0`.

H. Timer1 Configuration

Timer1 is the 16-bit hardware timer that handles all the timing. Here's how it's configured:

Mode:

- CTC (Clear Timer on Compare Match) mode using the `WGM12` bit
- The timer automatically resets when it reaches the `OCR1A` value
- This lets us generate precise, repeating time intervals

Prescaler:

- Set to 1024 (using `CS12` and `CS10` bits)
- This divides the 16 MHz clock: $16 \text{ MHz} / 1024 = 15625 \text{ Hz}$
- Each timer tick is $64 \mu\text{s}$
- Maximum interval possible: $65536 \times 64 \mu\text{s} \approx 4.19 \text{ s}$

Calculation: To get the right number of timer ticks for a duration:

$$\text{ticks} = \frac{\text{duration}_{\text{ms}} \times 15625}{1000}$$

This value goes into `OCR1A`, and the interrupt fires when the timer counter (`TCNT1`) matches it.

I. Interrupt Service Routine

The `ISR(TIMER1_COMPA_vect)` function runs every time Timer1 triggers. It does:

- 1) **LED Update:** Sets the color and position for the current animation step
- 2) **Wave Effect:** Figures out which LEDs to light for the lighthouse effect
- 3) **Display:** Calls `led_strip.show()` to send the data to the LEDs
- 4) **Timer Reload:** Calculates and sets `OCR1A` for the next interval
- 5) **Next Step:** Moves to the next step (loops back to start when done)

Timing Impact: The WS2812B needs precise bit timing ($1.25 \mu\text{s}$ per bit, 24 bits per LED). For 8 LEDs, `show()` takes about $240 \mu\text{s}$. Since the shortest Morse interval is 500 ms, this $240 \mu\text{s}$ overhead is only 0.05% and doesn't affect accuracy.

J. EEPROM State Management

The EEPROM is used to remember whether the system should be on or off across power cycles. Byte address 0 stores:

- **Value 0:** System stopped
- **Value 1:** System playing

How it works: When you press reset, the `setup()` function:

- 1) Reads the current state from EEPROM
- 2) Flips it (on becomes off, off becomes on)
- 3) Writes the new state back (only if it changed)
- 4) Shows the startup or shutdown light sequence

Protecting EEPROM: EEPROM can only handle about 100,000 writes before wearing out. To avoid wasting writes, the code only writes to EEPROM when the value actually changes.

K. Visual Feedback Sequences

The system provides visual confirmation of state transitions:

Startup Sequence: Red (500 ms) → Green (500 ms) → Off (500 ms) → Begin transmission

Shutdown Sequence: Green (500 ms) → Red (500 ms) → Off → Stay off

These sequences provide immediate feedback about the system state and help debug power or timing issues.

L. Implementation Challenges

Several technical challenges came up during development:

WS2812B Timing Conflicts: The WS2812B protocol requires very precise timing (within $\pm 150 \text{ ns}$). Initially, I worried that calling `show()` inside an ISR might cause timing conflicts or jitter. However, since the shortest Morse interval (500 ms) is much longer than the LED update time ($240 \mu\text{s}$), this wasn't an issue in practice.

Array Bounds Checking: Early versions occasionally had LEDs lighting up at wrong positions due to integer overflow in the wave effect calculation. Adding explicit boundary checks (`if (ledPos >= 0 && ledPos < LED_COUNT)`) fixed this completely.

EEPROM Wear: The original design wrote to EEPROM on every reset. Since EEPROM has limited write cycles, I added a check to only write when the value actually changes. This simple optimization extended the theoretical lifespan from a few months (assuming 100 resets/day) to over 270 years.

Timer Overflow: When calculating timer ticks, I initially used 16-bit arithmetic which caused overflow for longer intervals. Switching to 32-bit arithmetic (`uint32_t`) for the intermediate calculation solved this: `((uint32_t) durations[currentStep] * TICKS_PER_SECOND) / 1000UL`.

IV. TESTING AND VALIDATION

A. Timing Accuracy

Timer1 with a 1024 prescaler provides timing resolution of $64 \mu\text{s}$ per tick. For a 500 ms dot:

$$\text{Error} = \frac{64 \mu\text{s}}{500 \text{ ms}} = 0.0128\%$$

This precision far exceeds human perception thresholds, ensuring imperceptible timing drift even over extended operation.

B. Message Verification

I tested the system with several different messages:

- **"IN":** Simple two-letter example (shown in the video)
- **"SOS":** The classic emergency signal (... — ...)
- **"SAMUEL MORSE":** A longer word to test letter and word spacing

All messages encoded correctly with the right timing.

C. State Persistence

Reset button functionality was verified through multiple power cycles:

- 1) Initial state: Stopped
- 2) First reset: Starts playing
- 3) Second reset: Stops playing
- 4) Third reset: Resumes playing
- 5) ...

EEPROM state correctly persisted across power disconnections, confirming non-volatile storage functionality.

D. Wave Effect Variations

I tested the lighthouse effect with different `WAVE_WIDTH` values:

- **WAVE_WIDTH = 0:** Standard mode - all LEDs on/off together
- **WAVE_WIDTH = 1:** Single LED sweeps across (narrow beam)
- **WAVE_WIDTH = 3:** Three LEDs illuminate (medium beam)
- **WAVE_WIDTH = 5:** Five LEDs illuminate (wide beam) - default setting
- **WAVE_WIDTH = 7:** Very wide beam, almost fills the entire strip

The `WAVE_WIDTH = 5` setting provided the best balance between visual smoothness and distinctiveness of the sweeping effect.

This confirms the implementation is stable for extended operation, as would be expected from an interrupt-driven design.

V. ALTERNATIVE APPROACHES

A. Software vs. Hardware Timing

An alternative implementation could use software delays (`delay()`) instead of interrupts:

- **Pros:** Simpler code and easier to understand. Furthermore, this approach would also allow for a nice pulsing mechanism, to slowly tune the intensity of each LED up and down again.
- **Cons:** Blocks the main loop, prevents multitasking, less accurate timing

The interrupt approach was mainly chosen because it keeps the main loop free and provides deterministic timing regardless of other code execution. Besides, this course is about Embedded Systems, so using Hardware Timers or Interrupts is mandatory.

B. Alternative LED Technologies

Other LED options considered:

- **Standard LEDs:** Much simpler to control, but no individual addressing or color control
- **APA102:** Similar to WS2812B but with separate clock/data lines, more forgiving timing
- **LED Matrix:** Could display actual text, but significantly more complex

WS2812B was chosen for its balance of capability (RGB, individual control) and simplicity (single data line).

C. Message Input Methods

For dynamic message input, several options exist:

- **Serial Interface:** Simple text input via USB
- **Bluetooth:** Wireless control from smartphone
- **SD Card:** Store multiple pre-recorded messages
- **Physical Keyboard:** Matrix keypad for standalone operation

The current compile-time message was chosen for simplicity, but adding serial input would be straightforward (see Section VI-B).

VI. DISCUSSION

A. Limitations

Current limitations include:

- Fixed message stored in program memory (requires re-compilation to change)
- Limited to 100 steps (approximately 10-15 characters), but limit can be increased.
- No real-time input mechanism
- Unidirectional communication (transmit only)
- No error detection or correction

B. Future Enhancements

Cool improvements that could benefit this project include:

- Serial interface for dynamic message input
- Multiple color schemes for different message types
- Variable speed control via potentiometer
- Sound output via piezo buzzer
- Message recording and playback from EEPROM
- A full setup with multiple devices (at least two), where each has at least

- All features described in this project, plus
- External input keyboard
- Photodiode receiver for receiving signals
- Function to translate incoming morse signals to strings
- Display to show message string

would allow users to exchange morse messages in bidirection.

C. Lessons Learned

This project provided valuable insights into embedded systems development:

Plan for Interrupts Early: Deciding to use interrupts from the start simplified the architecture. Retrofitting interrupt support into a `delay()`-based design would have been much harder.

Document Timing Requirements: Keeping clear notes on timing requirements (WS2812B protocol, Morse code standards, ISR execution time) helped avoid conflicts and made debugging easier.

Test Incrementally: Building the project in stages (basic LED control → timer interrupts → Morse encoding → wave effect) made it easier to isolate bugs. Each stage worked before adding the next feature.

Memory is Limited: The ATmega328P's 2 KB RAM seems like a lot until you start using arrays. Planning memory usage upfront and using flash storage for constants was essential.

Comments Matter: Documenting the code with Doxygen-style comments helped me understand my own code when debugging issues days later. The time spent writing comments was worth it.

VII. CONCLUSION

This project successfully implements an interrupt-driven Morse code transmitter with visual output. It demonstrates several important embedded systems concepts: hardware timer configuration, interrupt service routines, non-volatile memory management, and addressable LED control.

Using Timer1 in CTC mode keeps the timing accurate regardless of what else the program is doing. The EEPROM state management gives a simple way to toggle on/off using just the reset button. The lighthouse wave effect makes it visually interesting and shows precise control over individual LEDs.

Main accomplishments:

- Very accurate timing (sub-millisecond precision)
- Clean, modular code that's easy to extend
- Efficient use of memory
- An empty main loop

This project is a good foundation for learning about real-time embedded systems and could be easily extended with features described in VI-B.

DECLARATION OF AI USAGE

I hereby declare that I used AI tools to assist in the generation of code and text for this project. However, I have reviewed and verified all content and take responsibility for the accuracy and integrity of this report and the associated software to the same extent as if I had written it entirely myself.

VIII. APPENDIX: CODE

```

1 // Arduino: Nano Atmel mega328p
2 // LED: WS2812B (CJMCU-2812-8)
3 // Used: Timer1 CTC, prescaler 1024, Button interrupt
4
5 #include <Arduino.h>
6 #include <Adafruit_NeoPixel.h> // LED
7 #include <EEPROM.h> // State persistence (button press handling)
8 // Message
9 const char MESSAGE[] = "IN"; //< change this to whatever message you want
10
11 // Pin setup
12 const uint8_t LED_PIN = 6; //< (Digital) Pin of LED stripe
13 const uint8_t WAVE_WIDTH = 5; //< set to 0 for normal blinking, >0 for lighthouse effect
14 const uint8_t LED_COUNT = 8; //< total number of LEDs in the strip
15
16 // Morse Code Timing Configuration (in milliseconds)
17 const uint16_t TIME_DOT = 500; //< Duration of a dot
18 const uint16_t TIME_DASH = 1500; //< Duration of a dash
19 const uint16_t TIME_SYMBOL_GAP = 500; //< Gap between dots/dashes within a character
20 const uint16_t TIME_LETTER_GAP = 1500; //< Gap between characters
21 const uint16_t TIME_WORD_GAP = 3500; //< Gap between words
22 const uint8_t BRIGHTNESS = 1; //< Brightness in % from 0 to 100%, currently 1% very dim
23
24 // Init an instance called led_strip of Adafruit_NeoPixel class
25 Adafruit_NeoPixel led_strip(LED_COUNT, LED_PIN, NEO_GRB + NEO_KHZ800);
26
27 // Timing in seconds, scaled to Timer1 ticks each ISR reload
28 const uint16_t TICKS_PER_SECOND = 15625; //< 16MHz / 1024 = 15625
29 const uint8_t MAX_STEPS = 100; //< Sufficient for most messages
30
31 // arrays to store message
32 uint16_t durations[MAX_STEPS]; //< duration in milliseconds per step
33 uint32_t colors[MAX_STEPS]; //< RGB color for each step (packed as 0xRRGGBB)
34 uint8_t ledIndex[MAX_STEPS]; //< Index of center LED at each time step in [0;LED_COUNT] with
    255 = all LEDs
35 uint8_t stepCount = 0; //< Total number of steps in the sequence
36 volatile uint8_t currentStep = 0; //< Current step index being executed
37
38 /**
39  * @brief Lookup table for Morse symbols.
40  *
41  * Converts a single character into its Morse code representation
42  * using dots ('.') and dashes ('-').
43  *
44  * @param c The character to convert (case-insensitive handling done by caller).
45  * @return const char* String representing the Morse code, or empty string if unknown.
46  */
47 const char* morseCode(char c) {
48     switch (c) {
49         case 'A': return ".-";
50         case 'B': return "-...";
51         case 'C': return "-.-.";
52         case 'D': return "-..";
53         case 'E': return ".";
54         case 'F': return ".-..";
55         case 'G': return "--";
56         case 'H': return "....";
57         case 'I': return "..";
58         case 'J': return ".---";
59         case 'K': return "-.-";
60         case 'L': return ".-..";
61         case 'M': return "--";
62         case 'N': return "-.";
63         case 'O': return "---";
64         case 'P': return ".-.-";
65         case 'Q': return "--.-";
66         case 'R': return ".-.";
67         case 'S': return "...";
68         case 'T': return "-";
69         case 'U': return "...";
70         case 'V': return "...-";
71         case 'W': return "-.-";
72         case 'X': return "-.-.";
73         case 'Y': return "-.-.-";
74         case 'Z': return "--..";

```

```

75     case '0': return "-----";
76     case '1': return ".----";
77     case '2': return "..---";
78     case '3': return "...--";
79     case '4': return "....-";
80     case '5': return ".....";
81     case '6': return "-....";
82     case '7': return "--...";
83     case '8': return "---..";
84     case '9': return "----.";
85     default: return "";
86 }
87 }
88
89 /**
90  * @brief Adds a single step to the Morse sequence.
91  *
92  * Records the duration and color for a specific part of the sequence.
93  *
94  * @param duration_ms Duration of the step in milliseconds.
95  * @param color RGB color (use led_strip.Color() or 0 for off).
96  * @param ledIdx LED index to light (255 = all LEDs).
97  */
98 void addStep(uint16_t duration_ms, uint32_t color, uint8_t ledIdx = 255) {
99     if (stepCount < MAX_STEPS) {
100         durations[stepCount] = duration_ms;
101         colors[stepCount] = color;
102         ledIndex[stepCount] = ledIdx;
103         stepCount++;
104     }
105 }
106
107 /**
108  * @brief Constructs the full Morse code sequence for the global MESSAGE.
109  *
110  * Converts each character to Morse code and creates a lighthouse wave effect
111  * where the light sweeps across all LEDs for each dot/dash.
112  * Adds appropriate gaps between symbols, letters, and words.
113  * Includes a final wave effect at the end before looping.
114  */
115 void buildSequence() {
116     stepCount = 0;
117     const uint32_t WHITE = led_strip.Color(255, 255, 255);
118     const uint32_t GREEN = led_strip.Color(0, 255, 0); // not used right now
119     const uint32_t RED = led_strip.Color(255, 0, 0); // not used right now
120     const uint32_t OFF = 0;
121
122     // loop through the message
123     for (const char* p = MESSAGE; *p; ++p) {
124         char c = toupper(*p);
125         if (c == ' ') {
126             addStep(TIME_WORD_GAP, OFF); // space between words
127             continue;
128         }
129         const char* code = morseCode(c);
130         if (!*code) continue; // skip unknown characters
131
132         // go through each dot or dash
133         for (const char* s = code; *s; ++s) {
134             uint16_t onDur = (*s == '.') ? TIME_DOT : TIME_DASH;
135             uint16_t perLedDur = onDur / LED_COUNT;
136
137             // lighthouse wave - light moves across the strip
138             for (uint8_t i = 0; i < LED_COUNT; i++) {
139                 addStep(perLedDur, WHITE, i);
140             }
141
142             if (*(s + 1)) {
143                 addStep(TIME_SYMBOL_GAP, OFF); // gap between dots/dashes
144             }
145         }
146         addStep(TIME_LETTER_GAP, OFF); // gap between letters
147     }
148     // End of message
149     addStep(TIME_WORD_GAP, OFF);
150
151     // cool reverse wave at the end before looping

```

```

152     for (uint8_t i = LED_COUNT; i > 0; i--) {
153         addStep(20, WHITE, i);
154     }
155     addStep(TIME_WORD_GAP, OFF);
156 }
157
158 /**
159  * @brief Timer1 Compare Match A Interrupt Service Routine.
160  *
161  * Updates the LED state for the current step in the sequence.
162  * Handles both full strip control and wave effects with multiple LEDs.
163  * The sequence loops automatically when complete.
164  *
165  * Note: LED updates can block for ~300 microseconds (WS2812B protocol).
166  * This is acceptable given the slow Morse code timing (500ms+).
167  */
168 ISR(TIMER1_COMPA_vect) {
169     // Update LED state - either individual LED or all LEDs
170     if (ledIndex[currentStep] == 255 || WAVE_WIDTH == 0) {
171         // Light all LEDs (standard mode or wave disabled)
172         led_strip.fill(colors[currentStep]);
173     } else {
174         // wave effect with multiple neighbouring LEDs
175         led_strip.clear();
176         if (colors[currentStep] != 0) {
177             // Light up center LED and neighbours
178             int8_t center = ledIndex[currentStep];
179             int8_t halfWidth = WAVE_WIDTH / 2;
180
181             for (int8_t offset = -halfWidth; offset <= halfWidth; offset++) {
182                 int8_t ledPos = center + offset;
183                 // Only light up LEDs within strip boundaries -> else unexpected behaviour with leds on at wrong "
184                 // end" of strip (like index was taken modulo by number of leds in strip)
185                 if (ledPos >= 0 && ledPos < LED_COUNT) {
186                     led_strip.setPixelColor(ledPos, colors[currentStep]);
187                 }
188             }
189         }
190     }
191     led_strip.show();
192
193     // Calculate how many timer ticks for current step duration
194     uint32_t ticks = ((uint32_t)durations[currentStep] * TICKS_PER_SECOND) / 1000UL;
195     if (ticks < 100) ticks = 100; // minimum value
196
197     OCR1A = (uint16_t)(ticks - 1);
198
199     currentStep++;
200
201     // loop back to start
202     if (currentStep >= stepCount) {
203         currentStep = 0;
204     }
205 }
206
207 /**
208  * @brief Displays startup sequence: Red -> Green -> Off.
209  */
210 void showStartupSequence() {
211     led_strip.fill(led_strip.Color(255, 0, 0)); // Red
212     led_strip.show();
213     delay(500);
214     led_strip.fill(led_strip.Color(0, 255, 0)); // Green
215     led_strip.show();
216     delay(500);
217     led_strip.fill(0); // Off
218     led_strip.show();
219     delay(500);
220 }
221
222 /**
223  * @brief Displays shutdown sequence: Green -> Red -> Off.
224  */
225 void showShutdownSequence() {
226     led_strip.fill(led_strip.Color(0, 255, 0)); // Green
227     led_strip.show();

```



```

228     delay(500);
229     led_strip.fill(led_strip.Color(255, 0, 0)); // Red
230     led_strip.show();
231     delay(500);
232     led_strip.fill(0); // Off
233     led_strip.show();
234 }
235
236 /**
237  * @brief Arduino setup function.
238  *
239  * Toggles between playing and stopped states using EEPROM.
240  * Reset button press during play: restarts from beginning
241  * Reset button press when stopped: starts playing
242  */
243 void setup() {
244     led_strip.begin();
245
246     // Calculate and set brightness
247     uint8_t brightness = (255 * BRIGHTNESS) / 100;
248     led_strip.setBrightness(brightness);
249
250     // Using EEPROM to remember state between resets
251     // Pressing reset button toggles on / off
252
253     // Read state (0 = Stopped, 1 = Playing)
254     byte state = EEPROM.read(0);
255     state = !state; // toggle
256
257     // Actually EEPROM has ~100k write wear -> Only write if changed and thus really necessary to write
258     if (EEPROM.read(0) != state) {
259         EEPROM.write(0, state);
260     }
261
262     if (state) {
263         // Start messaging
264         showStartupSequence();
265
266         currentStep = 0;
267         buildSequence();
268
269         cli(); // disable interrupts
270         TCCR1A = 0;
271         TCCR1B = 0;
272         TCCR1B |= (1 << WGM12); // CTC mode
273         TCCR1B |= (1 << CS12) | (1 << CS10); // prescaler 1024
274         OCR1A = 1562; // ~100ms
275         TIMSK1 |= (1 << OCIE1A); // enable interrupt
276         sei(); // enable interrupts
277     } else {
278         // Stop messaging
279         showShutdownSequence();
280
281         // Disable Timer1 interrupt
282         TIMSK1 &= ~(1 << OCIE1A);
283         TCCR1B = 0; // Stop timer
284     }
285 }
286
287 /**
288  * @brief Main application loop.
289  *
290  * Empty - all Morse code logic is interrupt-driven via Timer1.
291  */
292 void loop() {
293     // main loop free for other tasks
294 }

```